

Some real-life applications of Greedy Algorithms

The Greedy Method

- An optimisation problem (OP) is a problem that involves searching through a set of configurations to find one that minimises or maximizes an objective function defined on these configurations
- The greedy method solves a given OP going through a sequence of (feasible) choices
- The sequence starts from well-understood starting configuration, and then iteratively makes the decision that seems best from all those that are currently possible.

Fractional Knapsack Problem

- In fractional knapsack problem, we are given a set S of n items, s.t., each item ' i ' has a positive benefit P_i and a positive weight W_i , and we wish to find the profit subset that doesn't exceed the total weight W of the knapsack.

Fractional Knapsack Problem

- Consider all the items with their weights and profits mentioned respectively.
- Calculate P_i/W_i of all the items and sort the items in descending order based on their P_i/W_i values.
- Without exceeding the limit, add the items into the knapsack.

- If the knapsack can still store some weight, but the weights of other items exceed the limit, the fractional part of the next time can be added.
- Hence, giving it the name fractional knapsack problem.

Fractional Knapsack Problem

- I.e., we can take an amount x_i of each item i such that

$$0 \leq x_i \leq w_i \text{ for each } i \in S \quad \text{and} \quad \sum_{i \in S} x_i \leq W.$$

- # The *total benefit* of the items taken is determined by the *objective function*

$$\sum_{i \in S} b_i(x_i/w_i)$$

Fractional Knapsack Problem

Items	1	2	3	4	5
Weights (in kg)	3	3	2	5	1
Profits	10	15	10	12	8

$$W_i = \{3, 3, 2, 5, 1\}$$

$$P_i = \{10, 15, 10, 12, 8\}$$

Step:-1 Calculate P_i/W_i for all the items

Items	1	2	3	4	5
Weights (in kg)	3	3	2	5	1
Profits	10	15	10	20	8
P_i/W_i	3.3	5	5	4	8

Step 2: Arrange all the items in descending order based on P_i/W_i

Items	5	2	3	4	1
Weights (in kg)	1	3	2	5	3
Profits	8	15	10	20	10
P_i/W_i	8	5	5	4	3.3

- Without exceeding the knapsack capacity, insert the items in the knapsack with maximum profit.

Knapsack = {5, 2, 3}

- However, the knapsack can still hold 4 kg weight, but the next item having 5 kg weight will exceed the capacity. Therefore, only 4 kg weight of the 5 kg will be added in the knapsack.

Items	5	2	3	4	1
Weights (in kg)	1	3	2	5	3
Profits	8	15	10	20	10
Knapsack	1	1	1	4/5	0

- Hence, the knapsack holds the weights =
- $[(1 * 1) + (1 * 3) + (1 * 2) + (4/5 * 5)] = 10,$
- with maximum profit of $[(1 * 8) + (1 * 15) + (1 * 10) + (4/5 * 20)] = 49.$

Job Sequencing with Deadlines:

The sequencing of jobs on a single processor with deadline constraints is called as Job Sequencing with Deadlines.

Here-

- You are given a set of jobs.
- Each job has a defined deadline and some profit associated with it.
- The profit of a job is given only when that job is completed within its deadline.

	J1	J2	J3
Profit	250	100	200
Deadline	2	2	1

Assumptions-

- Only one processor is available for processing all the jobs.
- Processor takes one unit of time to complete a job.
- Non-preemption is there.

The problem states-

“How can the total profit be maximized if only one job can be completed at a time?”

Approach to Solution-

- A feasible solution would be where each job in the subset gets completed within its deadline.
- Value of the feasible solution would be the sum of profit of all the jobs contained in the subset.
- An optimal solution of the problem would be a feasible solution which gives the maximum profit.

Steps to solve-

Step-01:

- Sort all the given jobs in decreasing order of their profit.

Step-02:

- Check the value of maximum deadline.
- Draw a Gantt chart where maximum time on Gantt chart is the value of maximum deadline.

Step-03:

- Pick up the jobs one by one.
- Put the job on Gantt chart as far as possible from 0 ensuring that the job gets completed before its deadline.

Example Problem-

Given the jobs, their deadlines and associated profits as shown, answer the following questions-

- Write the optimal schedule that gives maximum profit.
- What is the maximum earned profit?

Jobs	J1	J2	J3	J4	J5	J6
Deadlines	5	3	3	2	4	2
Profits	200	180	190	300	120	100

Solution-

Step1: Sort all the given jobs in decreasing order of their profit-

Jobs	J4	J1	J3	J2	J5	J6
Deadlines	2	5	3	3	4	2
Profits	300	200	190	180	120	100

Step2:

Value of maximum deadline = 5.

So, draw a Gantt chart with maximum time on Gantt chart = 5 units as shown-



Gantt Chart

Now,

- We take each job one by one in the order they appear in Step1.
- We place the job on Gantt chart as far as possible from 0.

Step3:

- We take job J4.
- Since its deadline is 2, so we place it in the first empty cell before deadline 2 as-



Step4:

- We take job J1.
- Since its deadline is 5, so we place it in the first empty cell before deadline 5 as-



Step5:

- We take job J3.
- Since its deadline is 3, so we place it in the first empty cell before deadline 3 as-



Step6:

- We take job J2.
- Since its deadline is 3, so we place it in the first empty cell before deadline 3.
- Since the second and third cells are already filled, so we place job J2 in the first cell as-



Step7:

- Now, we take job J5.
- Since its deadline is 4, so we place it in the first empty cell before deadline 4 as-



Now,

- The only job left is job J6 whose deadline is 2.
- All the slots before deadline 2 are already occupied.
- Thus, job J6 can not be completed.

The optimal schedule is-

J2 , J4 , J3 , J5 , J1

This is the required order in which the jobs must be completed in order to obtain the maximum profit.

Maximum earned profit

= Sum of profit of all the jobs in optimal schedule

= Profit of job J2 + Profit of job J4 + Profit of job J3 + Profit of job J5 + Profit of job J1

= 180 + 300 + 190 + 120 + 200

= 990 units

Time Complexity Analysis:

Time for sorting on basis of profits $= O(n \log n)$

Activity Selection Problem

- Suppose $S = \{1, 2, \dots, n\}$ is the set of n proposed activities. The activities share resources which can be used by only one activity at a time, e.g., Tennis Court, Lecture Hall, etc.
- Each Activity "i" has **start time** s_i and a **finish time** f_i , where $s_i \leq f_i$. If selected activity "i" take place meanwhile the half-open time interval $[s_i, f_i)$.
- A new activity can only be selected if previous selected activity has completed
- Objective is to select activities in such a way that maximum number of activities can be performed

Given a set of 10 activities and their start and end time.
Find the list of activities selected.

$$S = (A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8, A_9, A_{10})$$

$$S_i = (1, 2, 3, 4, 7, 8, 9, 9, 11, 12)$$

$$f_i = (3, 5, 4, 7, 10, 9, 11, 13, 12, 14)$$

- 1. Arrange activities in ascending order of their finish time
- 2. A new activity can only be selected if previous selected activity has completed
- Selected activities:- A1, A3, A4, A6, A9, A10

GREEDY- ACTIVITY SELECTION (s, f)

1. $n \leftarrow \text{length } [s]$ // activities are arranged in ascending order of f.t.
2. $A \leftarrow \{1\}$
3. $j \leftarrow 1$ // j is the index of last activity selected
4. for $i \leftarrow 2$ to n
5. do if $s_i \geq f_j$
6. then $A \leftarrow A \cup \{i\}$
7. $j \leftarrow i$
8. return A

Huffman coding

- Huffman coding is a popular algorithm used for lossless data compression. It assigns variable-length codes to input characters, with shorter codes assigned to more frequent characters, and longer codes assigned to less frequent characters.
- This ensures that the most common characters are represented using fewer bits, leading to overall compression of the data.

Steps involved

- **1. Frequency Counting:**
- The first step is to analyze the input data and count the frequency of occurrence of each character. This step can be performed by iterating through the input data and building a frequency table that stores the count of each character.

- **Building the Huffman Tree:**

- Once the frequency counts are obtained, the next step is to construct a Huffman tree. The tree is built in a bottom-up manner, starting with individual characters as leaf nodes.
- Each leaf node is assigned a weight equal to its frequency count.
- Then, iteratively combine the two nodes with the lowest weights into a new internal node. The weight of the new node is the sum of the weights of its children.
- This process continues till all characters are included in a single binary tree

- Assigning Huffman Codes:**

- After constructing the Huffman tree, traverse the tree from the root to each leaf node, assigning binary codes to each character.

- Assign a '0' to edges leading to the left child and a '1' to edges leading to the right child.

- The code for each character is the sequence of '0's and '1's encountered during the traversal from the root to that character's leaf node.

4. Encoding

- Once the Huffman codes are assigned, the input data can be encoded using these codes.
- Replace each character in the input data with its corresponding Huffman code.
- The encoded data will be a sequence of '0's and '1's representing the compressed version of the input.

5. Decoding

- To decode the encoded data, start from the root of the Huffman tree and traverse down the tree according to the encoded bits.
- Whenever a leaf node is reached, output the corresponding character and return to the root to continue decoding the next sequence of bits.
- Continue this process until all encoded bits are processed, resulting in the original input data.

Question

character	Frequency
a	5
b	9
c	12
d	13
e	16
f	45

- Continue this process until all nodes are combined into a single root node, forming a binary tree structure known as the Huffman tree.
- During this process, it's essential to maintain a priority queue (often implemented using a min-heap) to efficiently select the nodes with the lowest weights for combination.